

CHƯƠNG 9

BẢO MẬT & CỨNG CỐ HỆ THỐNG

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Hiểu được ý nghĩa của việc bảo mật hệ thống.
- Thiết lập Firewall và SSH Hardening để bảo vệ máy chủ.
- Sử dụng Fail2Ban chống tấn công brute-force.
- Thực hiện rà quét lỗ hổng trên server và container.

9.1 Đảm bảo an toàn hệ thống phần mềm

9.1.1 Ý nghĩa của việc bảo mật hệ thống

Trong kỷ nguyên chuyển đổi số hiện nay, bảo mật hệ thống máy chủ không còn là một công việc kỹ thuật đơn thuần mà đã trở thành nền tảng cốt lõi cho sự tồn vong của bất kỳ tổ chức nào. Ý nghĩa quan trọng nhất của bảo mật nằm ở việc duy trì ba trụ cột cốt lõi là tính bảo mật, tính toàn vẹn và tính sẵn sàng, hay còn gọi là bộ ba CIA. Tuy nhiên, trong môi trường thực tế hiện nay, những khái niệm này đã được mở rộng và trở nên phức tạp hơn rất nhiều. Tính bảo mật không chỉ dừng lại ở việc ngăn chặn truy cập trái phép mà còn liên quan mật thiết đến việc bảo vệ quyền riêng tư của người dùng theo các khung pháp lý mới như Luật Bảo vệ dữ liệu cá nhân của Việt Nam hoặc tiêu chuẩn NIS2 của Châu Âu. Khi một hệ thống bị xâm nhập, thiệt hại không chỉ là những con số tài chính hữu hình mà còn là sự sụp đổ của niềm tin từ khách hàng và đối tác, điều mà đôi khi phải mất nhiều thập kỷ mới xây dựng được.

Sự thay đổi quan trọng nhất trong tư duy bảo mật hiện đại là việc từ bỏ mô hình "lâu đài và hào nước" truyền thống – nơi chúng ta mặc định tin tưởng mọi thứ bên trong mạng nội bộ – để chuyển sang kiến trúc Zero Trust. Triết lý này dựa trên nguyên tắc "không bao giờ tin tưởng, luôn luôn xác thực", buộc mọi yêu cầu truy cập dù đến từ bên trong hay bên ngoài mạng đều phải được kiểm tra danh tính và ngữ cảnh một cách gắt gao. Việc áp dụng Zero Trust có ý nghĩa sống còn vì nó giúp hạn chế sự di chuyển ngang của kẻ tấn công bên trong hệ thống nếu chẳng may một mắt xích bị chiếm quyền. Điều này đặc biệt quan trọng trong bối cảnh làm việc từ xa và sử dụng thiết bị cá nhân trở nên phổ biến, khiến ranh giới của mạng doanh nghiệp không còn tồn tại một cách rõ ràng.

Bên cạnh đó, bảo mật hệ thống còn đóng vai trò then chốt trong việc đảm bảo hiệu suất vận hành và tối ưu hóa tài nguyên. Một máy chủ không được củng cố tốt có thể dễ dàng bị biến thành công cụ cho các hoạt động phi pháp như đào tiền điện tử (cryptojacking) hoặc tham gia vào các mạng botnet để thực hiện tấn công từ chối dịch vụ (DDoS) vào các mục tiêu khác. Khi đó, tài nguyên CPU, RAM và băng thông của doanh nghiệp bị chiếm dụng, gây ảnh hưởng trực tiếp đến chất lượng dịch vụ cung cấp cho người dùng hợp pháp. Do đó, việc triển khai các biện pháp bảo mật không chỉ là đề

chống lại hacker mà còn là để bảo vệ tài sản số và duy trì sự ổn định cho các hoạt động kinh doanh hàng ngày.

Cuối cùng, ý nghĩa của bảo mật hệ thống còn nằm ở khả năng chống chịu (resilience) trước các thảm họa dữ liệu. Với sự gia tăng của mã độc tống tiền (ransomware), bảo mật không chỉ là ngăn chặn mà còn là chuẩn bị cho kịch bản xấu nhất. Một hệ thống có cấu trúc bảo mật tốt sẽ đi kèm với các giải pháp sao lưu bất biến, đảm bảo rằng ngay cả khi kẻ tấn công đoạt quyền quản trị cao nhất, dữ liệu vẫn có thể được phục hồi mà không cần phải trả tiền chuộc. Đây chính là "tám khiên" cuối cùng giúp doanh nghiệp duy trì tính liên tục và khả năng sống sót trong một thế giới mạng đầy rủi ro.

9.1.2 Các nguy cơ gây mất an toàn hệ thống

Các nguy cơ đối với an toàn hệ thống trong giai đoạn hiện nay đã tiến hóa theo hướng tinh vi và khó phát hiện hơn, trong đó sai sót về cấu hình (misconfiguration) nổi lên như một điểm yếu chí mạng nhất. Thống kê từ các tổ chức an ninh mạng hàng đầu cho thấy có tới 79% các cuộc tấn công vào hệ thống Linux không sử dụng các loại mã độc phức tạp mà thay vào đó là khai thác các dịch vụ bị phơi bày ra Internet với cấu hình mặc định không an toàn. Chẳng hạn, việc để các dịch vụ như Redis hay MySQL lắng nghe trên địa chỉ IP công khai (0.0.0.0) mà không có tường lửa che chắn hoặc sử dụng mật khẩu yếu đã trở thành con đường ngắn nhất để hacker chiếm quyền điều khiển. Nguy cơ này càng trở nên nghiêm trọng khi các hệ thống ngày càng phức tạp, khiến các quản trị viên khó có thể bao quát hết mọi thiết lập an ninh.

Một nguy cơ khác đang trở thành tâm điểm của cộng đồng bảo mật chính là sự thất bại trong chuỗi cung ứng phần mềm (software supply chain failures). Thay vì tấn công trực tiếp vào mục tiêu, kẻ tấn công sẽ nhắm vào các thư viện mã nguồn mở, các gói thành phần bên thứ ba hoặc các công cụ phát triển mà doanh nghiệp đang sử dụng. Những vụ việc nổi tiếng như SolarWinds hay lỗ hổng Log4Shell đã minh chứng rằng một sơ hở nhỏ trong một thư viện phổ biến có thể tạo ra tác động dây chuyền tới hàng chục nghìn tổ chức trên toàn cầu. Đặc biệt, sự xuất hiện của các loại sâu máy tính nhắm vào kho lưu trữ npm như Shai-Hulud đã cho thấy xu hướng hacker đang tích cực đầu độc các công cụ mà lập trình viên tin dùng để cài cắm mã độc ngay từ giai đoạn phát triển.

Sự bùng nổ của trí tuệ nhân tạo (AI) cũng mang lại những nguy cơ mới mà trước đây chưa từng được tính đến. Trong khi AI giúp tăng tốc độ viết code, các nghiên cứu gần đây chỉ ra rằng có tới 45% mã nguồn do AI gợi ý chứa các lỗ hổng bảo mật nghiêm trọng như XSS hoặc tiêm nhiễm mã lệnh. Nguyên nhân là do AI thường được huấn luyện trên các bộ dữ liệu mã nguồn công khai có chất lượng không đồng đều, dẫn đến việc lặp lại các lỗi lập trình phổ biến. Ngoài ra, hiện tượng "ảo giác AI" còn khiến các công cụ này gợi ý sử dụng những thư viện không tồn tại, tạo cơ hội cho kẻ tấn công tạo ra các gói giả mạo trùng tên để lừa người dùng cài đặt (typosquatting). Điều này tạo ra một rủi ro tiềm tàng đối với các lập trình viên không chuyên khi họ quá tin tưởng vào công nghệ mà bỏ qua các bước kiểm tra an ninh cơ bản.

Cuối cùng, không thể không nhắc đến nguy cơ từ yếu tố con người và sự chậm trễ trong việc cập nhật các bản vá lỗi. Mặc dù các nhà sản xuất phần mềm như Microsoft hay các bản phân phối Linux thường xuyên phát hành các bản vá cho lỗ hổng zero-day, nhưng nhiều doanh nghiệp tại Việt Nam vẫn gặp khó khăn trong việc triển khai kịp thời do lo ngại về sự tương thích và thời gian ngừng hoạt động của hệ thống. Kẻ tấn công

thường tận dụng khoảng thời gian ”vàng” từ khi lỗ hổng được công bố cho đến khi hệ thống được vá để thực hiện các cuộc tấn công leo thang đặc quyền. Kết hợp với kỹ nghệ xã hội (social engineering) sử dụng công nghệ deepfake, hacker có thể dễ dàng lừa đảo nhân viên để lấy thông tin đăng nhập, biến con người thành mắt xích yếu nhất trong hệ thống phòng thủ kiên cố nhất.

Tấn công Ransomware. Hình thức tấn công này dự kiến sẽ tiếp tục gia tăng với sự tinh vi và phức tạp hoá cao hơn từ các nhóm tội phạm mạng. Hầu hết các cuộc tấn công ransomware ngày nay liên quan đến việc đánh cắp dữ liệu thương mại cá nhân hoặc nhạy cảm, làm tăng chi phí và độ phức tạp của các sự cố và tăng nguy cơ tổn hại về danh tiếng. Phân tích các tổn thất đáng kể trên mạng trong những năm gần đây cho thấy tỷ lệ các trường hợp liên quan đến đánh cắp dữ liệu đã tăng gấp đôi, từ 40% lên gần 80% và xu hướng này dự kiến sẽ tiếp tục.

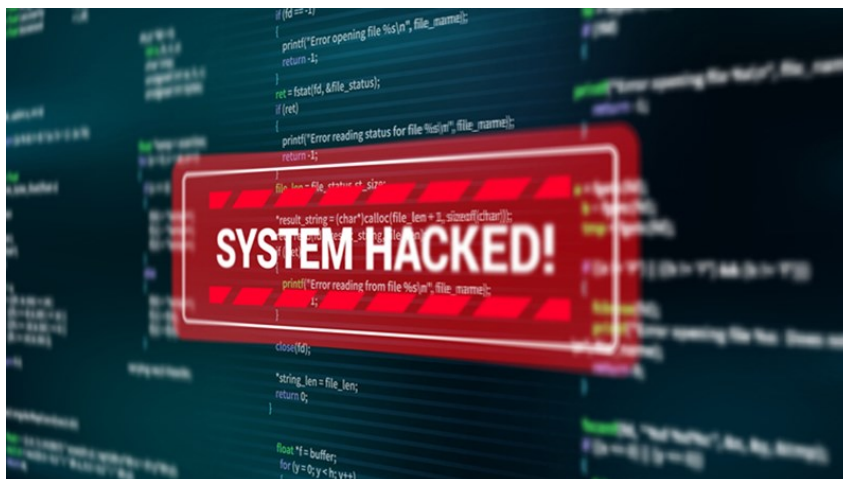
Tấn công vào các hệ thống ICS và OT/IT. Theo báo cáo của Microsoft, trong số 78% thiết bị IoT tồn tại các lỗ hổng đã biết trên hệ thống của khách hàng thì có tới 46% lỗ hổng không thể vá. Ngoài ra 25% thiết bị công nghệ vận hành (OT) trên hệ thống của khách hàng không được hỗ trợ. Nếu trước kia hệ thống OT/IT được thiết kế hoạt động riêng biệt thì ngày nay hệ thống điều khiển công nghiệp (ICS) đã bắt đầu kết nối với hệ thống công nghệ thông tin (IT), nên rủi ro tấn công vào hệ thống này đang trở nên lớn hơn. Các hệ thống ICS và OT thường không được vá lỗi và bị lộ nên đây là mục tiêu của tin tặc. Trong khi việc vá lỗi là thách thức của các tổ chức vì các bản cập nhật có thể chưa được thực hiện ngay bởi việc làm gián đoạn hoạt động. Có những thiết bị thiếu bản vá hoặc do ngừng hỗ trợ nên tin tặc hoàn toàn có thể khai thác các thiết bị này để tấn công bằng cách sử dụng các công cụ tìm kiếm có sẵn trên Internet để tìm được các cổng sử dụng quản lý từ xa và giành quyền truy cập trái phép.

Tấn công vào chuỗi cung ứng. Một trong những mối lo ngại lớn về rủi ro trong chuỗi cung ứng là việc sử dụng các thành phần của bên thứ ba như phần mềm, phần cứng được triển khai trong mạng OT hay hệ thống ICS của doanh nghiệp. Bởi các thành phần này không được các tổ chức nhận biết nên có thể gây ra mối đe dọa đáng kể về an ninh mạng, đặc biệt là những mạng không có thiết bị cách ly theo các quy định cụ thể. Kể từ năm 2020, các cuộc tấn công bảo mật vào chuỗi cung ứng phần mềm như SolarWinds, Log4j, Codecov và Kaseya đã ảnh hưởng tới hơn 490 triệu khách hàng và hơn 100.000 gói nguồn mở độc hại. Với mức tăng trung bình là 742% trong các cuộc tấn công mã nguồn mở kể từ năm 2019, con số này vẫn đang tiếp tục gia tăng.

Tấn công đa vector. Sự đa dạng của các kỹ thuật tấn công cho phép tội phạm mạng sử dụng các loại kỹ thuật tấn công khác nhau khi nhắm đến mục tiêu bao gồm tấn công cả bên trong lẫn bên ngoài, tấn công kỹ nghệ xã hội và cả các lỗ hổng zero-day.

Tấn công liên quan đến AI và ML. Hình thức tấn công này đang tiềm ẩn nhiều nguy cơ cao khi việc ứng dụng trí tuệ nhân tạo (AI) và học máy (ML) đang dẫn phát triển trên toàn cầu. Tội phạm mạng có thể tấn công vào nguồn dữ liệu nhằm nhiễm độc dữ liệu đào tạo cho AI/ML, dẫn đến việc mô hình máy học đưa ra các dự đoán sai lệch hoặc không chính xác. Hay phổ biến hơn là hình thức tấn công giả mạo Deepfake khi sử dụng AI để tạo ra các nội dung giả mạo như video, giọng nói hoặc hình ảnh để lừa đảo người xem. Cao cấp hơn là tấn công trực diện vào chính hộp đen (black box) của AI bằng cách sử dụng thông tin đầu ra của mô hình AI/ML để phá vỡ an ninh từ hoạt động bên trong.

Tấn công vào môi trường điện toán đám mây. Khi cloud trở nên phổ biến hơn, nó đồng nghĩa với việc trở thành đối tượng được quan tâm hơn đối với tin tặc. Bởi môi



Hình 9.1: Minh họa một hệ thống bị tấn công (system hacked)

trường điện toán đám mây hiện nay là nơi lưu trữ dữ liệu, mạng và các dịch vụ quan trọng của nhiều tổ chức. Các loại tấn công vào môi trường điện toán đám mây phổ biến là tấn công từ chối dịch vụ phân tán (DDoS), tấn công Application Programming Interface (API), Firewall Evasion, Man-in-the-Middle (MitM), tấn công đánh cắp dữ liệu, lây nhiễm mã độc, tấn công đám mây lai,...

Tấn công xâm phạm BEC. BEC là loại lừa đảo tinh vi nhằm vào doanh nghiệp có mối quan hệ hợp tác với các nhà cung cấp nước ngoài cũng như doanh nghiệp thường xuyên thực hiện việc thanh toán chuyển khoản. Đây là hình thức tấn công lừa đảo tinh vi nhằm vào tài chính của doanh nghiệp và dự kiến sẽ tiếp tục gia tăng.

9.1.3 Một số giải pháp phòng chống tấn công máy chủ dịch vụ

Trong danh mục các kỹ thuật tấn công máy chủ hiện nay, tiêm nhiễm SQL (SQL Injection - SQLi) vẫn đứng đầu về mức độ phổ biến và tác động nghiêm trọng. Bản chất của cuộc tấn công này là việc kẻ tấn công lợi dụng việc thiếu kiểm soát dữ liệu đầu vào để chèn các đoạn mã SQL độc hại vào các truy vấn của ứng dụng. Khi mã nguồn thực hiện phép cộng chuỗi để tạo câu lệnh truy vấn mà không sử dụng tham số hóa, câu lệnh của attacker như ' OR 1=1 -- sẽ khiến logic kiểm tra luôn đúng, cho phép họ vượt qua xác thực hoặc trích xuất toàn bộ cơ sở dữ liệu. Để phòng chống hiệu quả, giải pháp bắt buộc là sử dụng Parameterized Queries (Prepared Statements), giúp tách biệt hoàn toàn giữa phần lệnh và phần dữ liệu, khiến cho các ký tự đặc biệt của hacker không thể thay đổi được cấu trúc của câu lệnh gốc.

Mã độc tống tiền (ransomware) đã tiến hóa sang thế hệ mới với chiến thuật "tống tiền kép", không chỉ mã hóa dữ liệu mà còn đe dọa rò rỉ các thông tin nhạy cảm của doanh nghiệp lên mạng nếu không nhận được tiền. Các nhóm tội phạm hiện nay thường sử dụng kỹ thuật BYOVD (Bring Your Own Vulnerable Driver) để nạp các trình điều khiển có lỗ hổng vào hệ thống, từ đó chiếm quyền truy cập mức kernel và vô hiệu hóa các phần mềm bảo mật (EDR) trước khi tiến hành mã hóa. Phương án phòng vệ tối ưu nhất là triển khai mô hình Defense-in-Depth (phòng thủ theo chiều sâu) kết hợp với Immutable Backup. Dữ liệu sao lưu phải được lưu trữ dưới dạng "Write Once, Read Many" (WORM), đảm bảo rằng ngay cả khi hacker chiếm được quyền admin cao nhất, họ cũng không thể xóa hay sửa đổi các bản sao lưu này.

Tấn công dò tìm mật khẩu (brute force) vẫn là một mối đe dọa thường trực đối với các công dịch vụ như SSH. Các mạng botnet tự động sẽ liên tục rà soát cổng 22 trên toàn bộ dải IP công cộng để thử hàng triệu tổ hợp tên người dùng và mật khẩu phổ biến. Để đối phó, quản trị viên cần thực hiện việc củng cố hệ thống (hardening) bằng cách vô hiệu hóa đăng nhập bằng mật khẩu và thay thế bằng xác thực khóa công khai (SSH key) chuẩn Ed25519. Việc kết hợp thêm xác thực đa yếu tố (MFA) và các công cụ phòng thủ cộng đồng như CrowdSec sẽ giúp chặn đứng các địa chỉ IP độc hại ngay từ khi chúng có ý định gửi yêu cầu truy cập đầu tiên dựa trên dữ liệu chia sẻ từ hàng triệu máy chủ khác trên thế giới.

Một hình thức tấn công đang gia tăng mạnh mẽ là tấn công dựa trên định danh (identity-based attacks) nhắm vào các môi trường Cloud và SaaS. Ví dụ điển hình là vụ rò rỉ dữ liệu của Snowflake năm 2024, nơi hacker không cần bẻ khóa hệ thống mà chỉ cần sử dụng các thông tin đăng nhập đánh cắp được từ máy tính cá nhân của nhân viên để truy cập thẳng vào kho dữ liệu trên mây. Do các cuộc tấn công này không để lại dấu vết trên các máy trạm do tổ chức quản lý, các giải pháp bảo mật truyền thống thường bị bỏ qua. Giải pháp cho vấn đề này là triển khai kiến trúc XDR (Extended Detection and Response) để giám sát các hành vi bất thường xuyên suốt từ mạng, định danh đến các dịch vụ đám mây, giúp phát hiện sớm các hành vi như đăng nhập từ vị trí bất thường hay các thay đổi quyền truy cập trái phép.

Case study: vụ rò rỉ dữ liệu Snowflake (2024) và bài học về quản trị định danh.

Trong năm 2024, hơn 165 tổ chức lớn (bao gồm AT&T và Ticketmaster) đã trở thành nạn nhân của một chiến dịch tấn công nhắm vào nền tảng dữ liệu đám mây Snowflake. Kẻ tấn công không khai thác lỗ hổng kỹ thuật trong phần mềm của Snowflake mà thay vào đó sử dụng các tài khoản quản trị không bật xác thực đa yếu tố (MFA). Các thông tin đăng nhập này bị thu thập từ mã độc đánh cắp thông tin (Infostealer) trên máy tính cá nhân của nhân viên tổ chức.

Nguyên nhân gốc rễ: sự tin tưởng mù quáng vào mật khẩu truyền thống và sự thiếu hụt trong giám sát định danh trên môi trường Cloud.

Bài học thực tiễn: bảo mật máy chủ hiện đại không chỉ là bảo vệ ”phần mềm” mà phải tập trung vào quản trị ”danh tính người dùng” và áp dụng nguyên tắc đặc quyền tối thiểu.

Các nguyên tắc đảm bảo an toàn cho hệ thống:

- **Bảo mật là một quá trình liên tục (process)**, không phải là một sản phẩm mua về là xong. Hệ thống cần được giám sát và cập nhật không ngừng.
- **Tư duy ”giả định bị xâm nhập” (assume breach)**: luôn chuẩn bị các lớp phòng thủ bên trong để cô lập thiệt hại khi lớp biên giới bị xuyên thủng.
- **Sự kết hợp giữa con người – quy trình – công nghệ**: công nghệ mạnh đến đâu cũng sẽ thất bại nếu con người không có nhận thức tốt và quy trình vận hành lỏng lẻo.

9.2 Triển khai bảo mật cho hệ thống máy chủ

9.2.1 Thiết lập cấu hình cho tường lửa trên máy chủ Linux (iptables)

Netfilter là một framework (bộ khung) cốt lõi được tích hợp sẵn trong nhân hệ điều hành Linux (Linux Kernel). Chức năng chính của nó là quản lý, kiểm soát, lọc dữ liệu

(packet) và thực hiện biên dịch địa chỉ mạng (NAT) trên hệ thống.

Netfilter cung cấp cho các module trong hệ điều hành khả năng can thiệp vào vòng đời của các gói tin mạng khi chúng đi vào hoặc đi ra khỏi máy tính. Nó đảm nhận các công việc sau:

- **Packet filtering (lọc gói tin):** chặn hoặc cho phép các gói tin đi qua.
- **NAT (Network Address Translation):** thay đổi địa chỉ IP/port của gói tin để định tuyến hoặc ẩn IP mạng nội bộ.
- **Connection tracking (theo dõi kết nối):** ghi nhớ trạng thái của các kết nối để tạo tường lửa thông minh.
- **Packet mangling:** chỉnh sửa thông tin, tiêu đề (header) của gói tin.

Netfilter hoạt động dựa trên các điểm đánh chặn (gọi là hooks) nằm trên luồng xử lý mạng (network stack) của Linux. Khi một gói tin chạy qua, nó sẽ kích hoạt các quy tắc được lập trình tại các hook:

- **PREROUTING:** xử lý gói tin ngay khi vừa nhận vào, trước khi tiến hành định tuyến.
- **INPUT:** gói tin được gửi trực tiếp đến hệ thống cục bộ.
- **FORWARD:** gói tin đi qua hệ thống (không dành cho máy hiện tại).
- **OUTPUT:** gói tin bắt nguồn từ chính hệ thống gửi đi.
- **POSTROUTING:** xử lý gói tin sau khi đã định tuyến và chuẩn bị đẩy ra ngoài card mạng.

Bản thân Netfilter là mã nguồn chạy ngầm bên dưới nhân Linux và không có giao diện cho người dùng. Để thiết lập các quy tắc cho Netfilter, chúng ta cần sử dụng các công cụ cấu hình ở không gian người dùng (userspace):

- **iptables:** công cụ dòng lệnh truyền thống và rất phổ biến trên các hệ điều hành Linux cũ và một số distro hiện nay.
- **nftables:** công cụ kế nhiệm hiện đại của iptables, cung cấp khả năng phân loại và quản lý quy tắc linh hoạt, hiệu năng cao hơn.

Trong các hệ thống mạng, tường lửa là thành phần phòng thủ lớp mạng quan trọng nhất, đóng vai trò như một người gác cổng kiểm soát mọi luồng dữ liệu ra vào máy chủ. Trong hệ sinh thái Linux, sức mạnh này được cung cấp bởi khung Netfilter bên trong nhân (kernel): iptables là công cụ quản trị truyền thống được sử dụng để định nghĩa các quy tắc (rules) cho Netfilter, giúp phân loại, sửa đổi hoặc ngăn chặn các gói tin dựa trên địa chỉ IP, cổng (port) và giao thức. Hiện nay, nftables đã trở thành chuẩn mặc định mới nhờ hiệu năng cao và cú pháp hợp nhất, nhưng iptables vẫn tồn tại phổ biến trong các hệ thống cũ (legacy) và là cơ sở để người học hiểu về logic lọc gói tin.

Triết lý thiết kế tường lửa: "Default Drop" và giảm thiểu bề mặt tấn công. Một sai lầm phổ biến của các quản trị viên mới là thiết lập tường lửa theo hướng "chặn những gì xấu" (blacklisting). Tuy nhiên, tư duy bảo mật hiện đại yêu cầu áp dụng nguyên tắc Default Drop (danh sách trắng – whitelisting). Theo triết lý này, máy chủ sẽ mặc định từ chối tất cả các kết nối mạng và chỉ mở đúng những cổng dịch vụ cần thiết cho công việc. Cách tiếp cận này giúp thu hẹp tối đa bề mặt tấn công, khiến các mạng botnet rà quét tự động không thể tìm thấy các dịch vụ nội bộ nhạy cảm đang chạy ngầm.

Trước khi triển khai, quản trị viên cần xác định rõ danh sách các dịch vụ đang hoạt động. Một máy chủ web điển hình thường chỉ cần mở cổng 80 (HTTP), 443 (HTTPS)

cho công chúng và cổng SSH (mặc định là 22 hoặc cổng tùy chỉnh) cho quản trị viên. Tất cả các cổng khác như MySQL (3306), Redis (6379) phải được khóa chặt khỏi thế giới bên ngoài và chỉ cho phép truy cập cục bộ hoặc qua kênh VPN an toàn.

Cấu trúc và logic vận hành của iptables. iptables hoạt động dựa trên cấu trúc các bảng (tables), trong đó mỗi bảng chứa các chuỗi (chains) và mỗi chuỗi chứa các quy tắc (rules). Bảng quan trọng nhất cho mục đích bảo mật là bảng *filter*, bao gồm ba chuỗi mặc định:

- **INPUT:** kiểm soát các gói tin đi vào máy chủ. Đây là nơi thiết lập hầu hết các quy tắc bảo mật.
- **OUTPUT:** kiểm soát các gói tin đi ra từ máy chủ. Việc siết chặt chuỗi này giúp ngăn chặn máy chủ bị biến thành trạm trung chuyển tấn công người khác nếu chẳng may bị nhiễm mã độc (như ngăn chặn các kết nối Command & Control).
- **FORWARD:** kiểm soát các gói tin đi xuyên qua máy chủ (thường dùng khi máy chủ đóng vai trò router hoặc gateway).

Mỗi quy tắc trong iptables sẽ kiểm tra gói tin từ trên xuống dưới. Nếu gói tin khớp với một quy tắc, hành động (target) tương ứng như ACCEPT (cho phép), DROP (hủy gói tin âm thầm) hoặc REJECT (từ chối và báo phản hồi) sẽ được thực thi và quá trình kiểm tra dừng lại.

Hướng dẫn thực hành: xây dựng tường lửa an toàn. Dưới đây là các bước cụ thể để thiết lập một bức tường lửa vững chắc trên môi trường Linux.

Bước 1: thiết lập chính sách mặc định (default policies). Trước khi bắt đầu, chúng ta cần đặt chính sách nghiêm ngặt nhất cho chuỗi INPUT. Lưu ý: luôn đảm bảo đã cho phép kết nối SSH hiện tại trước khi chạy lệnh DROP mặc định để tránh tự khóa mình khỏi hệ thống.

```
# Cho phép các kết nối hiện có không bị ngắt quãng
$sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
# Thiết lập chính sách mặc định: Chặn tất cả luồng vào và luồng chuyển tiếp
$sudo iptables -P INPUT DROP
$sudo iptables -P FORWARD DROP
# Cho phép máy chủ tự do gửi dữ liệu ra ngoài (có thể siết chặt sau)
$sudo iptables -P OUTPUT ACCEPT
```

Bước 2: cho phép các giao diện mạng thiết yếu. Nguyên tắc thường dùng đó là mọi máy chủ đều cần giao diện lặp (loopback) để các tiến trình nội bộ giao tiếp với nhau.

```
$sudo iptables -A INPUT -i lo -j ACCEPT
```

Bước 3: mở cổng cho các dịch vụ cụ thể. Chúng ta sẽ mở cổng SSH và các dịch vụ Web. Giả sử cổng SSH của bạn đã được đổi sang 2849 để tăng tính bảo mật.

```
# Mở cổng SSH cho một địa chỉ IP quản trị cụ thể (Khuyến dùng)
$sudo iptables -A INPUT -p tcp -s 123.45.67.89 --dport 2849 -j ACCEPT
# Mở cổng HTTP/HTTPS cho tất cả người dùng truy cập
$sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
$sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

Bước 4: cấu hình chống tấn công từ chối dịch vụ (DDoS) cơ bản. Chúng ta có thể sử dụng mô-đun `limit` hoặc `hashlimit` để ngăn chặn việc gửi gói tin quá nhanh từ một nguồn, giúp giảm thiểu tác động của tấn công brute-force hoặc ICMP flood.

```
# Giới hạn số lượng gói tin ICMP (ping) để tránh bị tràn ngập
$sudo iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 1/s -j ACCEPT
# Chống tấn công SYN Flood bằng cách giới hạn kết nối TCP mới
$sudo iptables -A INPUT -p tcp --syn -m limit --limit 1/s --limit-burst 3 -j ACCEPT
```

Sự chuyển dịch sang nftables và khả năng tương thích. Các chuyên gia khuyến nghị dần chuyển sang nftables. nftables cho phép thực hiện nhiều hành động trong một quy tắc duy nhất (ví dụ: vừa ghi log vừa chặn) và xử lý hàng nghìn quy tắc nhanh hơn iptables gấp nhiều lần nhờ cấu trúc dữ liệu tối ưu.

Ví dụ, để chuyển đổi một quy tắc từ iptables sang nftables, bạn có thể dùng công cụ `iptables-translate`:

```
# Lệnh iptables gốc
$iptables-translate -A INPUT -p tcp --dport 22 -j ACCEPT
# Kết quả nftables tương ứng
# nft add rule ip filter INPUT tcp dport 22 counter accept
```

Mặc dù cú pháp nftables dễ đọc hơn và hỗ trợ tốt cho cả IPv4 lẫn IPv6 trong cùng một khung (`inet`), việc nắm vững logic của iptables vẫn giúp chúng ta có cái nhìn sâu sắc về cách thức các gói tin được lọc ở tầng nhân hệ điều hành.

Case study: củng cố máy chủ web tại một doanh nghiệp thương mại điện tử. Giả sử một doanh nghiệp tại Việt Nam vận hành hệ thống web trên Ubuntu 24.04. Sau khi bị tấn công rà quét chiếm quyền MySQL do để lộ cổng 3306 ra Internet, đội ngũ kỹ thuật đã triển khai tường lửa theo quy trình:

- **Phân tích:** nhận diện MySQL chỉ cần phục vụ ứng dụng chạy trên chính máy chủ đó (localhost).
- **Phương án triển khai:** thiết lập iptables chỉ mở cổng 443 và cổng SSH tùy chỉnh. Cổng 3306 được cấu hình chỉ ACCEPT từ giao diện `lo`.
- **Kết quả:** các nỗ lực rà quét cổng từ bên ngoài hoàn toàn bị chặn đứng (DROP), Hardening Index của hệ thống tăng thêm 15 điểm khi kiểm tra bằng Lynis (hãy tìm hiểu về công cụ này để bổ sung kiến thức về kiểm thử bảo mật).

Hướng dẫn thực hành: kiểm tra và lưu trữ quy tắc. Sau khi thiết lập, các quy tắc iptables sẽ mất đi khi khởi động lại máy. Chúng ta cần thực hiện:

- Kiểm tra các quy tắc đang chạy: `sudo iptables -L -n -v` (tham số `-v` giúp xem lượng dữ liệu đã bị chặn).
- Lưu trữ vĩnh viễn: cài đặt gói `iptables-persistent`.
- Thử nghiệm: thử kết nối từ một máy khác vào một cổng không được phép (ví dụ cổng 21 - FTP) và quan sát log hệ thống.

Lệnh lưu:

```
$sudo netfilter-persistent save
```

9.2.2 Cấu hình bảo mật cho SSH

Giao thức SSH (Secure Shell) đóng vai trò là "chìa khóa vạn năng" cho phép quản trị viên thiết lập kết nối an toàn để điều khiển máy chủ qua các mạng không tin cậy. Tuy nhiên, do đặc thù luôn mở để nhận kết nối, SSH thường xuyên phải hứng chịu các chiến dịch rà quét tự động từ các mạng botnet ngay khi máy chủ vừa có địa chỉ IP công khai. Thống kê cho thấy hơn 90% các cuộc tấn công SSH nhắm vào tài khoản root với mật khẩu yếu hoặc cấu hình mặc định. Vì vậy, bảo mật SSH là nhiệm vụ sống còn, yêu cầu sự kết hợp giữa kỹ thuật mã hóa hiện đại và các chính sách quản lý phiên nghiêm ngặt.

Chiến lược quản lý định danh: vô hiệu hóa truy cập root và mật khẩu. Nguyên tắc đầu tiên trong việc củng cố SSH là loại bỏ các mục tiêu dễ đoán nhất cho kẻ tấn công. Tài khoản root là mục tiêu số một vì nó nắm giữ đặc quyền cao nhất; việc cho phép đăng nhập trực tiếp qua root đồng nghĩa với việc hacker đã đi được 50% chặng đường nếu biết tên người dùng. Quản trị viên nên cấu hình để hệ thống từ chối mọi yêu cầu đăng nhập bằng root và thay thế bằng việc sử dụng một tài khoản thông thường có quyền sudo. Điều này tạo ra một lớp bảo vệ bổ sung, buộc kẻ tấn công phải đoán trúng cả tên người dùng ẩn và mật khẩu hoặc khóa của tài khoản đó.

Bên cạnh đó, việc sử dụng mật khẩu truyền thống đã trở nên lỗi thời và cực kỳ nguy hiểm trước sức mạnh của các công cụ brute-force hiện đại. Thay vào đó, xác thực dựa trên khóa công khai (public key authentication) được coi là tiêu chuẩn vàng. Cơ chế này hoạt động dựa trên các cặp khóa mật mã phức tạp, khiến việc tấn công dò tìm trở nên bất khả thi về mặt toán học. Các chuyên gia khuyến nghị sử dụng chuẩn khóa Ed25519 thay cho RSA cũ vì kích thước nhỏ gọn nhưng cung cấp độ an toàn tương đương với RSA 4096-bit và tốc độ xử lý vượt trội.

Nâng cao hàng rào bảo vệ với xác thực đa yếu tố (MFA). Dù khóa SSH rất an toàn, rủi ro vẫn tồn tại nếu máy tính cá nhân của quản trị viên bị nhiễm mã độc đánh cắp thông tin (Infostealer) khiến khóa bí mật (private key) bị lộ. Để giải quyết điểm yếu này, việc tích hợp xác thực 2 yếu tố (2FA/MFA) là bắt buộc đối với các hệ thống production quan trọng. Bằng cách sử dụng các giao thức như TOTP (Time-based One-Time Password) qua Google Authenticator hoặc tin nhắn, hệ thống yêu cầu kẻ tấn công phải chiếm được cả tệp khóa và điện thoại của quản trị viên cùng lúc mới có thể xâm nhập. Lớp khóa bổ sung này giúp ngăn chặn hiệu quả các cuộc tấn công dựa trên định danh đang gia tăng mạnh mẽ hiện nay.

Kỹ thuật ẩn mình và giới hạn bề mặt tấn công. Một biện pháp đơn giản nhưng hiệu quả để giảm thiểu phiên toái từ các cuộc rà quét botnet là thay đổi cổng (port) mặc định của SSH từ 22 sang một số cổng ngẫu nhiên khác (ví dụ: 2849). Mặc dù hành động này không thể chặn đứng một hacker chuyên nghiệp có chủ đích rà soát toàn bộ cổng, nhưng nó giúp loại bỏ đến 99% các nỗ lực rà quét tự động hàng loạt, từ đó tiết kiệm tài nguyên CPU/RAM cho máy chủ và giữ cho tệp nhật ký (log) không bị phình to bởi các thông báo đăng nhập sai.

Hơn nữa, việc sử dụng tường lửa để giới hạn địa chỉ IP được phép truy cập SSH (whitelisting) là biện pháp kiểm soát biên mạnh mẽ nhất. Thay vì mở cổng SSH cho toàn bộ Internet, quản trị viên chỉ nên cho phép các kết nối từ dải IP của văn phòng hoặc thông qua một kênh VPN nội bộ. Cách tiếp cận này triệt tiêu hoàn toàn khả năng tiếp cận của các nguồn IP độc hại từ bên ngoài, biến cổng SSH trở nên "vô hình" đối với phần còn lại của thế giới mạng.

Tối ưu hóa cấu hình phiên và giám sát nhật ký. Để duy trì tính toàn vẹn của hệ thống, quản trị viên cần thiết lập các thông số quản lý phiên nghiêm ngặt trong tệp cấu hình. Các chính sách như Idle Timeout (tự động ngắt kết nối khi không hoạt động) giúp ngăn chặn các phiên đăng nhập bị bỏ quên trên các máy tính công cộng. Việc giới hạn số lượng nỗ lực đăng nhập sai qua các công cụ như `MaxAuthTries` hoặc giới hạn số lượng kết nối đồng thời qua `MaxStartups` cũng giúp làm chậm đáng kể quá trình tấn công của hacker.

Cuối cùng, mọi hoạt động của SSH phải được ghi lại một cách chi tiết để phục vụ công tác giám sát và điều tra sự cố. Việc thiết lập mức độ ghi nhật ký lên `VERBOSE` cung cấp đầy đủ thông tin về dấu vân tay của khóa được sử dụng và nguồn gốc kết nối. Quản trị viên thực chiến thường kết hợp các tệp nhật ký này với các giải pháp log tập trung hoặc hệ thống cảnh báo tự động qua Telegram/Email để phản ứng kịp thời ngay khi có dấu hiệu bất thường.

Case study: bài học từ sự cố chiếm quyền máy chủ production tại một startup. Trong một vụ tấn công thực tế, một startup đã cấp quyền SSH root cho toàn bộ đội ngũ kỹ thuật để thuận tiện trong việc bảo trì hệ thống. Tuy nhiên, do một thành viên sử dụng mật khẩu yếu, kẻ tấn công đã sử dụng công cụ Hydra để thực hiện brute-force thành công vào tài khoản root. Chỉ trong vòng vài giờ, toàn bộ dữ liệu của công ty đã bị mã hóa bởi ransomware và hacker đòi tiền chuộc để cấp mã giải khóa.

Sai lầm: bật root login, cho phép dùng mật khẩu và thiếu cơ chế giám sát log tập trung.

Khắc phục: sau sự cố, startup đã vô hiệu hóa root login, chuyển hoàn toàn sang SSH key Ed25519 có passphrase và triển khai xác thực 2 lớp (MFA) cho toàn bộ nhân viên. Kết quả là các nỗ lực tấn công brute-force đã giảm 95% và hệ thống đạt được tính ổn định tuyệt đối.

Thực hành bảo mật SSH: cấu hình SSH an toàn. Thực hiện tuần tự các bước sau để củng cố máy chủ Linux.

Bước 1: tạo cặp khóa SSH Ed25519 mạnh mẽ. Thực hiện trên máy cá nhân (client). Sử dụng câu lệnh sau đây để tạo cặp khóa với OpenSSH.

```
$ssh-keygen -t ed25519 -o -a 100 -C "admin@yourdomain.com"
```

Trong đó:

```
# -o: dùng định dạng OpenSSH mới
# -a 100: tăng số vòng KDF để chống brute-force khóa
# -C: thêm ghi chú định danh (email/thiết bị)
```

Sau đó, đẩy khóa công khai lên máy chủ:

```
$ssh-copy-id -p 22 <username>@<server_ip>
```

Bước 2: cấu hình tệp `/etc/ssh/sshd_config`. Mở tệp cấu hình bằng quyền sudo: `sudo nano /etc/ssh/sshd_config`. Chỉnh sửa hoặc thêm các dòng sau:

```
# Thay đổi cổng để tránh bot tự động
Port 2849
# Chỉ sử dụng giao thức phiên bản 2
Protocol 2
# Ngăn chặn đăng nhập root trực tiếp
PermitRootLogin no
# Chỉ cho phép đăng nhập bằng khóa, cấm mật khẩu
PasswordAuthentication no
PubkeyAuthentication yes
# Cấm mật khẩu trống
PermitEmptyPasswords no
# Ngắt kết nối sau 5 phút không hoạt động
ClientAliveInterval 300
ClientAliveCountMax 0
# Giới hạn nỗ lực đăng nhập
MaxAuthTries 3
# Tăng cường ghi log để giám sát
LogLevel VERBOSE
```

Bước 3: áp dụng thay đổi và kiểm tra. Trước khi restart, hãy kiểm tra cú pháp cấu hình:

```
$sudo sshd -t
```

Nếu không có lỗi, tiến hành restart dịch vụ:

```
$sudo systemctl restart ssh
```

Lưu ý: luôn giữ một kết nối SSH hiện tại đang mở cho đến khi bạn thử đăng nhập thành công ở một cửa sổ terminal mới qua cổng mới, tránh tình trạng tự khóa mình ra ngoài.

Một số nguyên tắc cần nhớ để bảo vệ hệ thống:

- **Chìa khóa thay cho mật khẩu:** SSH key Ed25519 là rào cản vững chắc nhất chống lại brute-force.
- **Nguyên tắc "mất gốc":** vô hiệu hóa đăng nhập root buộc kẻ địch phải thực hiện thêm bước leo thang đặc quyền, kéo dài thời gian để hệ thống phòng thủ phát hiện.
- **MFA là bắt buộc:** trong kỷ nguyên mã độc đánh cắp khóa, xác thực 2 lớp là tấm khiên cuối cùng bảo vệ định danh.
- **Ẩn mình thông minh:** đổi cổng và whitelist IP giúp giảm nhiễu hệ thống đáng kể.

9.2.3 Cấu hình giải pháp chống tấn công brute-force hệ thống

Tấn công dò tìm mật khẩu (brute-force) là hình thức tấn công thô sơ nhưng cực kỳ hiệu quả, dựa trên nguyên tắc thử và sai liên tục các tổ hợp tên đăng nhập và mật khẩu cho đến khi tìm thấy kết quả đúng. Theo các thống kê bảo mật mới nhất, hơn 90% các nỗ lực xâm nhập vào các dịch vụ quản trị như SSH nhắm vào tài khoản root hoặc các thông tin đăng nhập yếu. Nguy hiểm hơn, các cuộc tấn công này thường gây ra sự sụt

giảm hiệu suất nghiêm trọng cho máy chủ do chiếm dụng tài nguyên CPU và băng thông mạng để xử lý luồng yêu cầu khổng lồ. Vì vậy, một hệ thống củng cố (hardening) đạt chuẩn hiện đại không thể thiếu các cơ chế tự động nhận diện và chặn đứng (ban) các địa chỉ IP độc hại dựa trên hành vi của chúng.

Triết lý phòng thủ: từ cô lập thụ động đến phản ứng cộng đồng. Trong bối cảnh bùng nổ của các thiết bị IoT và AI, các cuộc tấn công brute-force đã thay đổi bản chất. Kẻ tấn công không còn dùng một IP duy nhất để thử hàng nghìn mật khẩu (dễ bị phát hiện) mà sử dụng hàng vạn IP khác nhau, mỗi IP chỉ thử một vài lần để né tránh các bộ lọc truyền thống. Điều này buộc các nhà quản trị hệ thống phải thay đổi tư duy: không chỉ bảo vệ máy chủ của mình một cách cô lập mà phải dựa vào dữ liệu đe dọa (threat intelligence) từ cộng đồng.

Giải pháp truyền thống như Fail2Ban hoạt động dựa trên việc quét tệp nhật ký (log), tìm kiếm các mẫu đăng nhập sai và ra lệnh cho tường lửa chặn IP đó trong một khoảng thời gian nhất định. Tuy nhiên, trong môi trường hiện đại, CrowdSec nổi lên như một bản nâng cấp vượt trội khi cho phép các máy chủ trên toàn thế giới chia sẻ danh sách các IP tấn công với nhau. Khi một máy chủ tại một nơi bất kỳ bị tấn công, thông tin này được đồng bộ về hệ thống trung tâm và gửi đến máy chủ của bạn để thực hiện chặn đứng "phòng ngừa" ngay cả khi kẻ tấn công chưa kịp gửi yêu cầu đầu tiên.

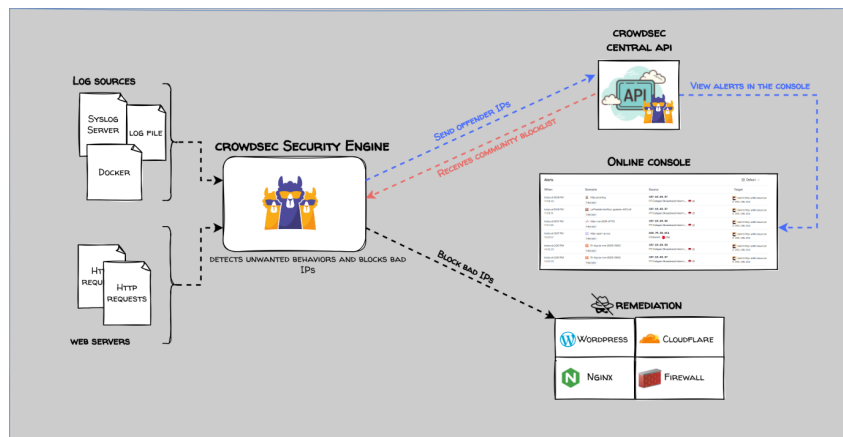
Giải pháp truyền thống: Fail2Ban và cơ chế lọc log. Fail2Ban là công cụ bảo mật phổ biến nhất cho Linux nhờ sự đơn giản và ổn định. Nó hoạt động như một "người giám sát" không mệt mỏi, liên tục rà soát các tệp log hệ thống (như `/var/log/auth.log` trên Ubuntu hoặc `/var/log/secure` trên CentOS). Khi số lượng nỗ lực đăng nhập thất bại từ một nguồn vượt quá ngưỡng cho phép (`MaxRetry`) trong một khoảng thời gian cụ thể (`FindTime`), Fail2Ban sẽ kích hoạt một quy tắc tường lửa (iptables/nftables) để từ chối mọi kết nối từ địa chỉ IP đó.

Mặc dù hiệu quả cho việc bảo vệ SSH cơ bản, Fail2Ban có một số hạn chế về hiệu suất do được viết bằng ngôn ngữ Python và sử dụng biểu thức chính quy (regex) để xử lý log, điều này có thể tạo ra nút thắt cổ chai nếu lượng log quá lớn. Ngoài ra, Fail2Ban mang tính chất phòng thủ thụ động – nó chỉ hành động sau khi máy chủ của bạn đã hứng chịu một lượng nỗ lực tấn công nhất định.

Giải pháp hiện đại: CrowdSec và sức mạnh của threat intelligence. CrowdSec được coi là tiêu chuẩn vàng cho việc chống brute-force trên các hệ thống production hiện nay. Được viết bằng ngôn ngữ Go với kiến trúc xử lý JSON-native, CrowdSec nhanh hơn Fail2Ban tới 60 lần và tiêu tốn ít tài nguyên hơn. Cấu trúc của CrowdSec gồm ba thành phần chính:

- **Acquisition:** thu thập log từ nhiều nguồn khác nhau (SSH, Nginx, MySQL...).
- **Parsers & Scenarios:** phân tích logic để nhận diện các kịch bản tấn công (không chỉ brute-force mà còn cả SQL Injection hay rà quét lỗ hổng).
- **Bouncers:** đây là các "lực lượng thực thi" – chúng có thể là một module tường lửa để chặn IP, một plugin Nginx để yêu cầu giải CAPTCHA, hoặc một module WAF để lọc các yêu cầu độc hại.

Điểm mạnh nhất của CrowdSec là hệ thống "Consensus". Mọi nỗ lực tấn công bị chặn trên máy chủ của bạn sẽ được gửi về Central API để đánh giá độ tin cậy. Nếu nhiều máy chủ cùng báo cáo một IP là độc hại, IP đó sẽ bị đưa vào "Community Blocklist" và bị chặn tự động trên hàng triệu hệ thống khác sử dụng CrowdSec toàn cầu.



Hình 9.2: Giải pháp CrowdSec chống tấn công

Hướng dẫn thực hành: triển khai giải pháp chống brute-force. Thực hiện các bước sau để thiết lập cả hai giải pháp (khuyến dùng CrowdSec cho hệ thống thực tế).

Phương án 1: cấu hình Fail2Ban (dành cho máy chủ tài nguyên thấp). Cài đặt và thiết lập ngưỡng chặn cho dịch vụ SSH:

```
# Cài đặt Fail2Ban
$ sudo apt install fail2ban -y
# Tạo tệp cấu hình tùy chỉnh để tránh bị ghi đè khi cập nhật
$ sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
# Chỉnh sửa tệp cấu hình
$ sudo nano /etc/fail2ban/jail.local
```

Trong tệp `jail.local`, tìm đoạn `[sshd]` và cấu hình:

```
[sshd]
enabled = true
port    = 2849          # Đổi thành cổng SSH tùy chỉnh của bạn
filter  = sshd
logpath = /var/log/auth.log
maxretry = 3            # Cho phép sai tối đa 3 lần
findtime = 600          # Trong vòng 10 phút
bantime  = 3600         # Chặn trong 1 giờ (có thể đặt -1 để chặn vĩnh viễn)
```

Sau đó khởi động lại dịch vụ:

```
$ sudo systemctl restart fail2ban
```

Phương án 2: cấu hình CrowdSec. Triển khai CrowdSec với module bảo vệ SSH:

```
# Cài đặt CrowdSec trên Ubuntu/Debian
$ curl -s https://packagecloud.io/install/repositories/crowdsec/crowdsec/script.deb.sh
↵ | sudo bash
$ sudo apt install crowdsec -y

# Cài đặt Firewall Bouncer (Thành phần thực hiện chặn IP trên tường lửa)
$ sudo apt install crowdsec-firewall-bouncer-iptables -y
```

Kiểm tra trạng thái các IP bị chặn và danh sách đe dọa từ cộng đồng:

```
# Xem các IP đang bị chặn trên máy của bạn
$sudo cscli decisions list
# Xem các danh sách đe dọa cộng đồng đang được đồng bộ
$sudo cscli alerts list
```

Lưu ý: nếu bạn chạy web server, bạn nên cài đặt thêm Nginx Bouncer để có thể hiển thị CAPTCHA thay vì chặn hoàn toàn, giúp tránh việc người dùng hợp pháp bị chặn nhầm.

So sánh Fail2Ban và CrowdSec:

Bảng 9.1: So sánh Fail2Ban và CrowdSec

Đặc điểm	Fail2Ban	CrowdSec
Ngôn ngữ	Python (chậm hơn)	Go (nhanh, hiệu năng cao)
Cơ chế phòng thủ	Cô lập (chỉ dựa trên máy chủ hiện tại)	Cộng đồng (chia sẻ dữ liệu toàn cầu)
Phạm vi bảo vệ	Chủ yếu Layer 3/4 (Firewall)	Đa tầng (Layer 7 CAPTCHA, WAF, Cloud)
Threat Intel	Không có	Rất mạnh (Community Blocklist)
Tài nguyên	Thấp, phù hợp VPS < 512MB RAM	Vừa phải, tối ưu cho hệ thống hiện đại

Case study: ngăn chặn mạng botnet 10.000 thiết bị tấn công startup. Một startup tại Việt Nam ghi nhận log SSH tăng vọt với hàng nghìn nỗ lực đăng nhập mỗi phút từ các dải IP trải dài khắp thế giới. Mỗi IP chỉ thử mật khẩu 1 lần rồi biến mất, khiến Fail2Ban truyền thống không thể kích hoạt lệnh chặn vì chưa đạt ngưỡng `maxretry`.

Giải pháp: đội ngũ kỹ thuật đã cài đặt CrowdSec.

Kết quả: ngay sau khi kích hoạt, CrowdSec đã tự động chặn đứng 80% các IP tấn công nhờ danh sách "Community Blocklist" có sẵn, dù các IP này chưa từng kết nối tới máy chủ của startup trước đó. Tải CPU của máy chủ giảm từ 90% xuống còn 5%, dịch vụ hoạt động ổn định trở lại.

Các khuyến cáo kỹ thuật:

- **Đừng phòng thủ đơn độc:** brute-force hiện đại dựa trên số lượng lớn IP, hãy sử dụng các giải pháp có threat intelligence như CrowdSec.
- **Giới hạn nỗ lực là chưa đủ:** ngoài việc chặn IP, cần kết hợp với việc đổi cổng SSH và dùng SSH key để triệt tiêu hoàn toàn khả năng thành công của hacker.
- **Giám sát log là bắt buộc:** dù dùng công cụ nào, việc ghi nhật ký chi tiết (`LogLevel VERBOSE`) là cơ sở để điều tra và cải thiện kịch bản bảo mật.
- **Cân bằng giữa bảo mật và vận hành:** khi thiết lập các ngưỡng chặn tự động, cần có cơ chế "whitelist" cho địa chỉ IP của văn phòng để tránh trường hợp quản trị viên tự khóa chính mình.

9.2.4 Phương án thử nghiệm phòng chống tấn công Brute-force

Trong thực tế, hơn 90% các cuộc tấn công SSH nhắm vào các thông tin đăng nhập yếu hoặc tài khoản mặc định, và kẻ tấn công thường sử dụng các công cụ tự động để quét

hàng nghìn mật khẩu trong thời gian ngắn. Việc thử nghiệm sẽ giúp chúng ta xác nhận xem tường lửa có thực sự chặn IP nguồn khi các nỗ lực đăng nhập sai vượt quá ngưỡng cấu hình hay không.

Phương án 1: thử nghiệm với công cụ Hydra (mô phỏng tấn công mạng chuyên nghiệp). Hydra được coi là một trong những công cụ bẻ khóa mật khẩu mạng nhanh nhất và linh hoạt nhất hiện nay, hỗ trợ hơn 50 giao thức bao gồm SSH, FTP, và HTTP. Công cụ này cho phép kẻ tấn công thực hiện các cuộc tấn công từ điển (dictionary attack) với tốc độ song song rất cao, khiến nó trở thành lựa chọn lý tưởng để thử nghiệm ngưỡng chịu tải và khả năng phản ứng của hệ thống phòng thủ.

Quy trình thực hiện: sinh viên chuẩn bị một tệp văn bản chứa danh sách các mật khẩu phổ biến (ví dụ: `passwords.txt` có sẵn trên Kali Linux) và thực hiện lệnh tấn công từ một máy trạm (client) hướng tới địa chỉ IP của máy chủ đã cài đặt Fail2Ban hoặc CrowdSec.

```
# -l: tên người dùng mục tiêu (ví dụ: admin)
# -P: đường dẫn tới tệp danh sách mật khẩu
# -t 4: số lượng luồng song song (giới hạn để tránh làm sập dịch vụ)
# ssh: giao thức mục tiêu
$hydra -l admin -P passwords.txt <server_ip> ssh -t 4
```

Trong kịch bản này, Hydra sẽ liên tục gửi các yêu cầu đăng nhập sai. Nếu Fail2Ban đã được cấu hình với tham số `maxretry = 3`, thì ngay sau lần thử thứ tư, địa chỉ IP của máy trạm sẽ bị chặn đứng bởi quy tắc tường lửa. Điều này chứng minh rằng cơ chế lọc log của Fail2Ban đã nhận diện được mẫu hành vi bất thường và ra lệnh thực thi cho iptables.

Phương án 2: thử nghiệm bằng scripting tự chế (mô phỏng botnet rải rác). Đối với các giải pháp hiện đại như CrowdSec, kẻ tấn công thường sử dụng các mạng botnet gồm hàng nghìn IP khác nhau, trong đó mỗi IP chỉ thử mật khẩu một vài lần để né tránh các bộ lọc tính dựa trên số lần thử sai đơn thuần. Để mô phỏng kịch bản này một cách đơn giản trên môi trường thực hành, sinh viên có thể sử dụng một vòng lặp Bash cơ bản để thực hiện các nỗ lực đăng nhập thử công từ một hoặc nhiều máy trạm khác nhau.

Quy trình thực hiện: thay vì dùng công cụ chuyên dụng, sinh viên có thể viết một đoạn script nhỏ để thực hiện 5–10 nỗ lực đăng nhập SSH với mật khẩu sai trong một khoảng thời gian ngắn.

```
for i in {1..5}; do
  ssh -o BatchMode=yes -o ConnectTimeout=5 <username>@<server_ip> 2>&1 | grep
  ↪ "Permission denied"
done
```

Điểm khác biệt ở đây là CrowdSec không chỉ dựa vào số lần thử sai mà còn phân tích các "kịch bản" (scenarios) tấn công dựa trên trí tuệ đe dọa cộng đồng. Ngay khi nhận thấy hành vi đăng nhập sai liên tục, CrowdSec sẽ ghi nhận một "quyết định" (decision) để chặn IP đó. Sinh viên cần kiểm tra xem CrowdSec có cập nhật danh sách đen dựa trên dữ liệu chia sẻ toàn cầu hay không, điều mà các công cụ truyền thống không làm được.

Quy trình kiểm tra và xác nhận kết quả. Sau khi thực hiện các cuộc tấn công giả lập trên, sinh viên cần tiến hành kiểm tra trên máy chủ để xác nhận tính hiệu quả của từng giải pháp thông qua các công cụ giám sát hệ thống.

Đối với Fail2Ban: sinh viên cần theo dõi tệp nhật ký an ninh tại `/var/log/auth.log` (trên Ubuntu) hoặc `/var/log/secure` (trên CentOS) để xem các thông báo lỗi đăng nhập. Sau đó, sử dụng lệnh của Fail2Ban để xác nhận IP đã bị "ban":

Lệnh kiểm tra:

```
$sudo fail2ban-client status sshd
```

Kết quả kỳ vọng: danh sách "Banned IP list" phải xuất hiện địa chỉ IP của máy trạm thực hiện tấn công.

Đối với CrowdSec: do CrowdSec hoạt động theo kiến trúc tách biệt giữa bộ phận phân tích và bộ phận thực thi (bouncer), sinh viên cần kiểm tra thông qua giao diện dòng lệnh `cscli`.

Lệnh kiểm tra:

```
$sudo cscli decisions list
```

Kết quả kỳ vọng: một quyết định chặn (`type: ban`) với lý do "ssh-bf" (SSH brute force) sẽ hiển thị kèm theo thời gian hết hạn của lệnh chặn. Sinh viên cũng có thể kiểm tra iptables bằng lệnh `sudo iptables -L -n` để thấy các tập hợp IP (ipset) do CrowdSec Bouncer tạo ra.

Câu hỏi thảo luận

Câu 1: Giải thích tại sao trong kiến trúc Zero Trust, vị trí mạng (trong hay ngoài công ty) không còn là yếu tố để xác định độ tin tưởng?

Câu 2: Tại sao việc sử dụng Parameterized Queries lại có thể ngăn chặn hoàn toàn tấn công SQL Injection dạng truyền thống?

Câu 3: Trong kịch bản tấn công BYOVD, tại sao các phần mềm diệt virus thông thường lại dễ dàng bị vô hiệu hóa?

Câu 4: Phân tích sự khác biệt về rủi ro giữa việc tự viết mã nguồn và việc sử dụng các thành phần bên thứ ba từ chuỗi cung ứng.

Câu 5:

- Giải thích sự khác biệt giữa hành động DROP và REJECT. Trong kịch bản bảo mật máy chủ công cộng, tại sao DROP thường được ưu tiên hơn?
- Viết một lệnh iptables để chỉ cho phép một địa chỉ IP duy nhất (ví dụ: 203.0.113.5) được phép SSH vào máy chủ.
- Tại sao việc cho phép interface `lo` (loopback) lại là bắt buộc đối với hầu hết các ứng dụng web hiện đại?